

Apellidos:

Nombre:

Concurrencia y Paralelismo

Bloque I: Concurrencia

Julio 2024

1. Cola supermercado [1.5 puntos]

Implementar el esquema de la cola de un supermercado donde existe una única cola de clientes y múltiples cajeros. Cuando un cajero queda libre escoge el primer cliente de la cola. Los cajeros y los clientes no pueden hacer espera activa. Se pueden añadir todos los campos necesarios a los struct customer y super. No se pueden usar variables globales.

```
struct customer {
    pthread_cond_t c;
    ...
}

struct super {
    queue *q;
    pthread_mutex_t *m;
    pthread_cond_t *cajeros;
    ...
};

void cashier(struct super *s) {
    struct customer *c;
    while(true) {
        ...
        serve_customer(c);
    }
}

void customer(struct super *s, int num_items) {
    ...
    pay_and_leave();
}

struct cliente *peek(queue *q); // Devuelve, sin eliminarlo, el primer cliente en la cola
void remove(queue *q, struct customer *c); // Quita el cliente indicado de la cola
void insert(queue *q, struct customer *c); // Inserta un cliente al final de la cola
```

```
int pthread_mutex_init(pthread_mutex_t *m, pthread_mutex_attr_t *attr);
int pthread_mutex_lock(pthread_mutex_t *m);
int pthread_mutex_trylock(pthread_mutex_t *m);
int pthread_mutex_unlock(pthread_mutex_t *m);
int pthread_cond_init(pthread_cond_t *c, pthread_cond_attr_t *attr);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mtx);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Solución:

```
struct customer {
    pthread_cond_t c;
};

struct super {
    queue *q;
    pthread_mutex_t m;
};

void cashier(struct super *s) {
    struct cliente *c;
    while(true) {
        lock(&s->m);
        while((c = peek(&s->q)) == NULL)
            wait(&s->cajeros, &s->m);

        pthread_cond_signal(c->c);
        remove(&s->q, c);
        unlock(&s->m);
        serve_customer(c);
    }
}

void customer(struct super *s, int num_items) {
    struct customer *c = malloc(sizeof(struct customer));
    pthread_cond_init(&c->c, NULL);

    pthread_mutex_lock(&s->m);
    insert(s->q, c);
    pthread_cond_signal(s->cajeros);
    pthread_cond_wait(&c->c, &s->m);
    pthread_mutex_unlock(&s->m);
    pay_and_leave();
}
```

2. Mutex con prioridades [2 puntos]

Implemente, utilizando los mutex de la librería pthread, un tipo de mutex donde se pueden hacer bloqueos con prioridad alta o baja. Cuando un mutex se libera, solo podrá ser bloqueado por un thread con prioridad baja si no hay ningún thread con prioridad alta esperando.

```
typedef {
    pthread_mutex_t m;
    pthread_cond_t c;
    ...
} prio_mutex;

void high_prio_lock(prio_mutex *m) {
    ...
}

void low_prio_lock(prio_mutex *m) {
    ...
}

void prio_unlock(prio_mutex *m) {
    ...
}
```

Implemente las operaciones high_prio_lock, low_prio_lock y prio_unlock.

Solución:

```
typedef {
    pthread_mutex_t m;
    pthread_cond_t c;
    int waiting_high_prio;
    int locked;
} prio_mutex;

void high_prio_lock(prio_mutex *m) {
    pthread_mutex_lock(&m->m);
    while(m->locked) {
        m->waiting_high_prio++;
        pthread_cond_wait(&m->c, &m->m);
        m->waiting_high_prio--;
    }
    m->locked = 1;
    pthread_mutex_unlock(&m->m);
}

void low_prio_lock(prio_mutex *m) {
    pthread_mutex_lock(&m->m);
    while(m->locked || m->waiting_high_prio > 0)
        pthread_cond_wait(&m->c, &m->m);

    m->locked=1;
    pthread_mutex_unlock(&m->m);
}

void prio_unlock(prio_mutex *m) {
    pthread_mutex_lock(&m->m);
    m->locked = 0;
    pthread_cond_broadcast(&m->c);
    pthread_mutex_unlock(&m->m);
}
```

3. Servidor de datos etiquetados [1.5 puntos]

Escriba un módulo que permita crear procesos servidor con la siguiente interfaz:

- `start()`, que arranca un proceso servidor y devuelve su PID.
- `put(S, Data, Tags)`, donde `S` es el PID de un proceso servidor, `Data` un dato arbitrario, y `Tags` una lista de etiquetas que describen el dato. El dato debe quedar almacenado en el servidor.
- `get(S, Tags)`, donde `S` es el PID de un proceso servidor, y `Tags` una lista de etiquetas. La función debe devolver una lista con todos los datos almacenados que tienen todas las etiquetas que están en `Tags`.

```
-module(store).
-export([start/0, get/2, put/3]).

start() ->
    spawn(?MODULE, init, []).
put(S, Data, Tags) ->
    ...
get(S, Tags) ->
    ...

init() ->
    ...
```

Por ejemplo:

```
1> S = store:start().
<0.10.0>
2> store:put(S, one, [green, large]).
ok
3> store:put(S, two, [small]).
ok
4> store:get(S, []).
[one, two]
5> store:get(S, [green]).
[one]
6> store:get(S, [red]).
[]
```

Solución:

```
-module(store).
-export([start/0, get/2, put/3]).

start() ->
    spawn(?MODULE, init, []).
put(S, Data, Tags) ->
    S ! {put, Data, Tags},
    ok.
get(S, Tags) ->
    S ! {get, Tags, self()},
    receive
        {get_reply, L} -> L
    end.

init() ->
    loop([]).

loop(L) ->
    receive
        {put, Data, Tags} ->
            loop([Data, Tags | L]);
        {get, Tags, From} ->
            Res = [Data || {Data, DTags} <- L, lists:all(fun(T) -> lists:member(T, DTags) end, Tags)],
            From ! {get_reply, Res},
            loop(L)
    end.
```
